

Enterprise Student Record & Academic Platform

Unit 1: Software Engineering, Design Thinking & Product Planning

Program: Careertiq Engineering Academic Management System (Agile & DevOps)
Student Name: Shravani Prakash Bankar
PRN: 2125UDSF1086

1. Introduction to Software Product Development

In modern higher education institutions, software platforms must be built using structured enterprise engineering frameworks rather than ad-hoc programming. Coding represents merely one phase within a larger engineering lifecycle spanning research, architecture, UX design, QA, automated deployment, and continuous observability.

1.1 Comparative Analysis: Programming vs. Software Product Engineering

Dimension	Programming / Coding	Software Product Engineering
Primary Goal	Translating business logic into executable code syntax.	Building scalable, secure, and maintainable academic management platforms.
Scope	Isolated scripts, single algorithms, or micro-functions.	Full SDLC, microservices architecture, DB schemas, security, and CI/CD.
Success Metric	Code compiles and passes basic unit tests.	Academic user adoption, system uptime, sub-second query performance, zero data loss.
Mindset	Technology-Centric ("How do I write this SQL query?").	Product-Centric ("How do we reduce student registration friction?").

1.2 Root Causes of Software Project Failures

- **Misaligned Requirements:** Building complex features that academic staff or students do not need.
- **Technology-Centric Bias:** Selecting frameworks based on industry hype rather than domain fit.
- **Communication Silos:** Separation between academic staff, developers, and DevOps infrastructure teams.
- **Lack of Iterative Feedback:** Delaying user testing with faculty and students until late development stages.

Software Development Lifecycle (SDLC) Workflow

- ```
1. Requirement Gathering & Domain Analysis
 ↓
2. Design Thinking & User Experience (UX) Wireframing
 ↓
3. System Architecture & Database Schema Design
 ↓
4. Implementation / Coding Phase (~20% Total Effort)
 ↓
```

5. Automated CI/CD, DevOps Pipeline & Testing

↓

6. Production Deployment, Observability & Loop

## 2. Business Understanding: Higher Education & Academic Management

Before engineering database models or writing backend logic, engineers must master the core operational workflows of academic institutions.

### 2.1 Core Operations in Academic Institutions

- **Student Onboarding & Registration:** Managing student admissions, profile generation, and program enrollment.
- **Course Enrollment & Scheduling:** Course credit verification, prerequisite checking, and timetable generation.
- **Attendance Management:** Real-time tracking of student presence with dynamic threshold notifications.
- **Examination & Evaluation:** Hall ticket generation, internal/external mark entry, and automated result processing.
- **Academic Analytics & Transcripts:** Automated SGPA/CGPA evaluation and official academic transcript publishing.

### 2.2 Manual Process Challenges vs. Automated Software Solutions

| Manual Operational Bottleneck                     | Automated Platform Solution                             | Business Outcome Metric                         |
|---------------------------------------------------|---------------------------------------------------------|-------------------------------------------------|
| Paper-based registration and physical filing      | Centralized digital onboarding portal & relational DB   | Onboarding time reduced from 3 days to <10 mins |
| Manual paper attendance registers kept by faculty | Real-time digital attendance marking & threshold alerts | Absence tracking accuracy raised to 99.9%       |
| Manual mark calculation & paper grade sheets      | Automated result processing & GPA engine                | Grade publication accelerated by 85%            |
| Uncoordinated records across departments          | Unified Role-Based Access Control (RBAC) database       | Data redundancy & errors reduced to 0%          |

## 3. Design Thinking Framework & Five Stages

Design Thinking provides a human-centered framework to solve complex institutional problems before committing to code.

[ EMPATHIZE ] → [ DEFINE ] → [ IDEATE ] → [ PROTOTYPE ] → [ TEST ]  
(Users & Pain)      (Problem)      (Solutions)      (Wireframes)      (Validation)

### Stage Breakdown:

- **Stage 1: Empathize:** Observational studies and interviews across Students, Faculty, Academic Administrators, and Exam Staff.
- **Stage 2: Define:** Formulated the core problem statement based on field observation.

CORE PROBLEM STATEMENT:  
"Students, faculty, and administrators experience operational delays, administrative overhead, and evaluation

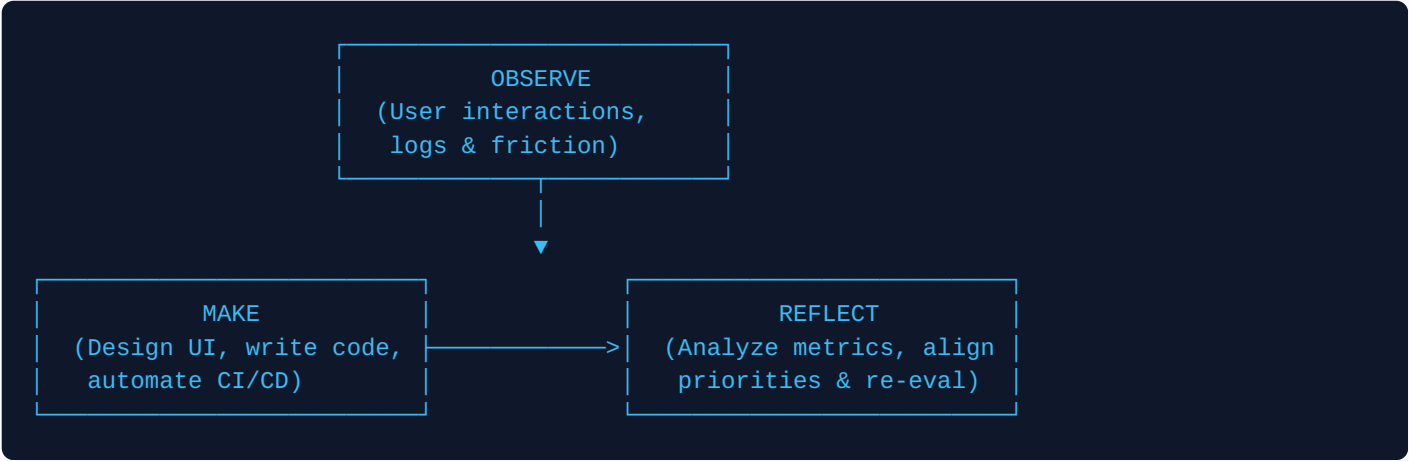
errors due to fragmented, manual record-keeping across admissions, course registration, attendance, and exam grading. Furthermore, leadership lacks real-time operational visibility."

- **Stage 3: Ideate:** Brainstormed key features: credit verification engines, automated mobile attendance, GPA calculators, and digital transcripts.
- **Stage 4: Prototype:** Built wireframes for Student Portal, Faculty Assessment Ledger, Exam Cell Hub, and Admin Dashboard.
- **Stage 5: Test:** Validated UI flows with faculty and students to optimize usability and task completion speed.

## 4. IBM Design Thinking & The IBM Loop Engine

Scales human-centered design for enterprise software delivery using three core pillars:

- **Focus on User Outcomes:** Shifting metric focus (e.g., "Enable faculty to mark class attendance in <30 seconds" instead of "Create attendance DB table").
- **Restless Reinvention:** Treating software continuous deployment (CD) as an ongoing refinement cycle.
- **Diverse Empowered Teams:** Cross-functional collaboration across Product Owners, Developers, QA, and DevOps.



## 5. Stakeholder Analysis & User Personas

### User Persona Profiles

| User Persona                                     | Primary Objectives                                                | Key Frustrations                                             | Target Platform Feature                               |
|--------------------------------------------------|-------------------------------------------------------------------|--------------------------------------------------------------|-------------------------------------------------------|
| <b>Priya</b><br>(Student)                        | Register for courses, track daily attendance, view GPA results.   | Opaque attendance tracking, delayed result announcements.    | Student Portal Dashboard with attendance gauge.       |
| <b>Prof. Sharma</b><br>(Faculty Member)          | Record attendance rapidly, submit assessment marks securely.      | Paper registers, manual grade calculation sheets.            | Express Mobile Attendance & Digital Mark Ledger.      |
| <b>Dr. Mehta</b><br>(Academic Admin / Exam Cell) | Verify student eligibility, process SGPA/CGPA, issue transcripts. | Interdepartmental data mismatch, manual transcript creation. | Automated Result Processing Engine & PDF Transcripts. |

Stakeholder Responsibility Matrix

| Stakeholder Group   | Key Roles                                 | Project Expectations                                    |
|---------------------|-------------------------------------------|---------------------------------------------------------|
| Business & Academic | Deans, Exam Officers, Faculty, Students   | Ensure academic compliance, ease of use, data accuracy. |
| Engineering         | Product Managers, Software Developers, QA | Architect clean codebase, secure RBAC, run test suites. |
| Operations          | DevOps Engineers, Cloud Architects, DBAs  | Ensure 99.9% uptime, automated CI/CD, data backups.     |

6. Requirements Engineering & MVP Definition

6.1 Functional Requirements Specification (FRS)

- **FR-01 Authentication & RBAC:** Secure login with role-based permissions (Admin, Faculty, Student, Exam Cell).
- **FR-02 Student Profile Management:** Full CRUD operations for student admission records and academic history.
- **FR-03 Course & Term Registration:** Prerequisite verification, credit limit enforcement, and course allocation.
- **FR-04 Attendance Engine:** Daily recording, percentage tracking, and automated shortage notifications.
- **FR-05 Examination & Result Engine:** Assessment entry, SGPA/CGPA calculation, and transcript generation.

6.2 Non-Functional Requirements Specification (NFRS)

- **NFR-01 Security:** Password hashing (Bcrypt), JWT session tokens, HTTPS, and full audit logging.
- **NFR-02 Performance:** Sub-500ms API response time under high concurrent user load.
- **NFR-03 Scalability & Uptime:** Containerized deployment targeting 99.9% availability with autoscaling.
- **NFR-04 Maintainability:** Infrastructure as Code (IaC) modularity and continuous integration testing.

PRODUCT VISION STATEMENT:

"To deliver a modern, cloud-native Student Record and Academic Management Platform that eliminates manual administrative overhead, accelerates evaluation workflows, and delivers real-time academic analytics."

MVP Release Scope (Phase 1)

Deferred non-essential capabilities (such as AI grade predictions or online fee gateways) to focus on key MVP features:

- User Login & Profile RBAC
- Student Onboarding & Course Allocation
- Attendance Tracking & Alert Triggering
- Assessment Mark Entry & Automated GPA Calculation
- PDF Academic Report & Transcript Generation

7. Enterprise Roadmap, Architecture & Repository Structure

7.1 Enterprise Roadmap Phases

1. **Phase 1:** Academic Domain Analysis & Business Requirements
2. **Phase 2:** Design Thinking, Wireframing & UX Prototyping

3. **Phase 3:** Agile Backlog Management & Architecture Planning
4. **Phase 4:** Application Development (Backend Services & Frontend Portal)
5. **Phase 5:** Automated Testing & DevOps CI/CD Integration
6. **Phase 6:** Infrastructure Provisioning & Security Hardening
7. **Phase 7:** Production Deployment, Observability & Feedback Loop

## 7.2 Repository Hierarchy & Structure

```
student-academic-platform/
├── .github/ # Automated CI/CD Pipelines (GitHub Actions)
├── app/ # Enterprise Application Source Code
├── modules/ # Business Logic (Admissions, Courses, Attendance, Exams)
├── database/ # Relational Schemas, Migrations, & Data Models
├── tests/ # Unit, Integration, & E2E Automated Tests
├── docs/ # Architecture Diagrams, BRD, FRS, & NFRS Specs
├── docker/ # Containerized Dockerfile & Compose Configurations
├── kubernetes/ # K8s Deployment Manifests & Ingress Rules
├── terraform/ # IaC Infrastructure Scripts
├── ansible/ # Configuration Management Rules
├── monitoring/ # Observability Dashboards (Prometheus / Grafana)
└── README.md # Platform Architecture & Developer Onboarding Guide
```

## Trainer Execution Deliverables Checklist

- ✓ Business Requirements Document (BRD) & Business Justification Report
- ✓ Product Vision Document & MVP Feature Matrix
- ✓ Stakeholder Analysis Report & User Persona Profiles
- ✓ Functional (FRS) and Non-Functional (NFRS) Requirements Specifications
- ✓ Design Thinking Report (Empathize, Define, Ideate, Prototype, Test)
- ✓ IBM Design Thinking Report & IBM Loop Execution Documentation
- ✓ Multi-Phase Enterprise Product Engineering Roadmap
- ✓ Initial System Architecture Diagram Concept Specs
- ✓ Project Repository Initialization & Standardized Folder Architecture